
Formal Specification of Signed Integer Representation and Arithmetic in Binary Computing

A Deep Dive into Complement Systems, IEEE 754, and Algorithmic Implementation

KOVALEV RUSLAN

Computer Systems Architecture Research

April 2025

Abstract

This research paper provides an exhaustive formalization of signed number representations. We move beyond basic definitions to explore the rigorous mathematical proofs underlying Two's Complement arithmetic and the IEEE 754 floating-point standard. The study includes detailed analyses of Sign-Magnitude, Biased representations, and hardware-level logic gates. Furthermore, we examine the complexities of overflow detection and provide a step-by-step derivation of Booth's recoding for signed multiplication.

Keywords: Two's Complement, IEEE 754, ALU, Binary Arithmetic, Booth's Algorithm, Overflow Detection

CONTENTS

1	Introduction	2
2	Theoretical Bounds of Representation	2
2.1	Sign-Magnitude (SM) Mapping	2
3	Biased Representation (Excess-K)	2
3.1	Formal Definition	2
4	Two's Complement: Mathematical Formalism	3
4.1	Definition by Weighted Sum	3
4.2	The Negation Proof	3
5	IEEE 754: Floating-Point Specification	3
5.1	Normalized Numbers	3
5.2	Subnormal (Denormalized) Numbers	4
6	Hardware Logic: Overflow Detection	4
6.1	Boolean Derivation	4
7	Advanced Signed Multiplication: Booth's Algorithm	4
7.1	Recoding Logic	4
7.2	Numerical Example	5
8	Software Perspective: Sign Extension	5
9	Conclusion	5
10	References	7

1 INTRODUCTION

The challenge of representing the set of integers $\mathbb{Z}$ and real numbers $\mathbb{R}$ within a finite set of binary states $\{0, 1\}^n$ is foundational to computer science. Unlike human notation, which uses a prefix symbol "-", digital hardware must embed the sign within the bit-string itself. This paper examines the trade-offs between various encoding schemes, focusing on their impact on the critical path of the Arithmetic Logic Unit (ALU).

2 THEORETICAL BOUNDS OF REPRESENTATION

For any n -bit representation, the total number of unique states is 2^n . The goal is to map these states to a range of values.

2.1 Sign-Magnitude (SM) Mapping

In SM, the MSB is the sign $s \in \{0, 1\}$, and the remaining $k = n - 1$ bits are the magnitude. The range is strictly symmetric:

$$Range_{SM} = [-(2^{n-1} - 1), 2^{n-1} - 1]$$

The value V of a bit vector B is:

$$V(B) = (-1)^{b_{n-1}} \sum_{i=0}^{n-2} b_i 2^i$$

The existence of "negative zero" ($1000 \dots 0_2$) causes a loss of efficiency and complicates hardware comparison logic.

3 BIASED REPRESENTATION (EXCESS-K)

Biased representation (or Offset Binary) maps integers by shifting the value by a fixed bias K . This is predominantly used in floating-point exponents.

3.1 Formal Definition

The value V is given by the unsigned interpretation of the bits minus the bias:

$$V = \left(\sum_{i=0}^{n-1} b_i 2^i \right) - K$$

Typically, for an n -bit exponent, $K = 2^{n-1} - 1$. **Example (8-bit, $K = 127$):**

$$\bullet \text{ } 1111111_2 \implies 255 - 127 = 128$$

- $01111111_2 \implies 127 - 127 = 0$
- $00000000_2 \implies 0 - 127 = -127$

4 Two's Complement: Mathematical Formalism

Two's Complement is the standard for integer arithmetic because it forms a ring isomorphism with $\mathbb{Z}_2^n$.

4.1 Definition by Weighted Sum

The MSB b_{n-1} has a negative weight:

$$V_{2c} = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$$

This asymmetry allows for exactly one more negative value than positive, eliminating the dual-zero problem:

$$Range_{2c} = [-2^{n-1}, 2^{n-1} - 1]$$

4.2 The Negation Proof

To prove that $NOT(x) + 1$ is the additive inverse:

Let $\bar{x}$ be the bitwise NOT of x . Then $x + \bar{x} = 2^n - 1$.

$$\bar{x} + 1 = 2^n - 1 - x + 1 = 2^n - x \equiv -x \pmod{2^n}$$

5 IEEE 754: Floating-Point Specification

The IEEE 754 standard defines how real numbers are represented using a sign bit, a biased exponent, and a mantissa.

5.1 Normalized Numbers

For most values, the hidden bit (implicit 1) is used:

$$V = (-1)^s \times (1.f) \times 2^{E-Bias} \tag{1}$$

Where f is the fractional part of the mantissa.

5.2 Subnormal (Denormalized) Numbers

When the exponent bits are all zero ($E = 0$), the number is subnormal, allowing for a graceful degradation to zero:

$$V_{sub} = (-1)^s \times (0.f) \times 2^{1-Bias} \quad (2)$$

Table 1: IEEE 754 Format Parameters

Precision	Total Bits	Exponent (E)	Mantissa (M)	Bias
Single (binary32)	32	8 bits	23 bits	127
Double (binary64)	64	11 bits	52 bits	1023

6 HARDWARE LOGIC: OVERFLOW DETECTION

Overflow (V) occurs when the result of an operation exceeds the representable range of n bits.

6.1 Boolean Derivation

For addition $S = A + B$, overflow occurs if two numbers of the same sign produce a result of the opposite sign:

$$V = A_{n-1}B_{n-1}\overline{S_{n-1}} + \overline{A_{n-1}}\overline{B_{n-1}}S_{n-1} \quad (3)$$

In optimized ALU design, this is equivalent to the XOR of carries:

$$V = C_n \oplus C_{n-1} \quad (4)$$

7 ADVANCED SIGNED MULTIPLICATION: BOOTH'S ALGORITHM

Booth's Algorithm recodes the multiplier to minimize the number of additions, handling signed integers natively.

7.1 Recoding Logic

The algorithm inspects the current bit Q_i and the previous bit Q_{i-1} (initially $Q_{-1} = 0$):

Q_i	Q_{i-1}	Operation
0	0	Shift only
0	1	$A \leftarrow A + M$, then shift
1	0	$A \leftarrow A - M$, then shift
1	1	Shift only

7.2 Numerical Example

Multiply $M = 3$ (0011_2) by $Q = -4$ (1100_2) in 4-bit space:

1. $A = 0000, Q = 1100, Q_{-1} = 0$.
2. Step 1 ($Q_0 Q_{-1} = 00$): Arithmetic Shift Right (ASR). $A = 0000, Q = 0110, Q_{-1} = 0$.
3. Step 2 ($Q_1 Q_0 = 00$): ASR. $A = 0000, Q = 0011, Q_{-1} = 0$.
4. Step 3 ($Q_2 Q_1 = 10$): $A \leftarrow A - M \implies 1101$. ASR. $A = 1110, Q = 1001, Q_{-1} = 1$.
5. Step 4 ($Q_3 Q_2 = 11$): ASR. $A = 1111, Q = 0100, Q_{-1} = 1$.

Result $AQ = 1110100_2$, which is -12 in 8-bit Two's Complement.

8 SOFTWARE PERSPECTIVE: SIGN EXTENSION

When promoting a signed m -bit integer to n bits, the sign bit must be replicated:

$$B = \underbrace{\{A_{m-1}, \dots, A_{m-1}, A_{m-1}, \dots, A_0\}}_{n-m \text{ times}}$$

```
#include <stdio.h>
#include <stdint.h>

int main() {
    int8_t a = 127;
    int8_t b = 1;
    int8_t sum = a + b; // Result: -128 (Overflow)

    int32_t extended = (int32_t)sum; // Sign extension: 0xFFFFF80
    printf("Sum: %d, Extended: %d\n", sum, extended);
    return 0;
}
```

Listing 1: C Example: Understanding Overflow and Promotion

9 CONCLUSION

The mathematical consistency of Two's Complement and the robustness of IEEE 754 form the backbone of modern computing. By unifying addition and subtraction logic and providing a formal path

for floating-point errors (NaN, Infinity), these systems ensure predictable behavior across diverse hardware architectures.

10 REFERENCES

1. Hennessy, J. L., & Patterson, D. A. (2011). *Computer Architecture: A Quantitative Approach*.
2. IEEE Computer Society. (2008). *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2008)*.
3. Stallings, W. (2015). *Computer Organization and Architecture*.